

Giảng viên ra đề: (Chữ ký và Họ tên)	(Ngày ra đề)	Người phê duyệt: (Chữ ký và họ tên)	(Ngày duyệt đề)
--	--------------	---	-----------------

 TRƯỜNG ĐH BÁCH KHOA - ĐHQG-HCM KHOA KH & KT MÁY TÍNH	THI CUỐI KỲ		Học kỳ / Năm học	1	2023-2024
			Ngày thi		24-12-2023
	Môn học	Cấu trúc dữ liệu & Giải thuật			
	Mã môn học	CO2003			
	Thời lượng	90 phút	Mã đề	2317	
Ghi chú: - Sinh viên không sử dụng tài liệu. Sinh viên nộp lại đề cùng với phiếu làm bài.					

I. Trắc nghiệm

- [L.O.2.1] Cho G là một đồ thị đơn với 20 đỉnh và 8 thành phần liên thông. Nếu xóa một đỉnh của G thì số thành phần liên thông của G sẽ nằm trong đoạn
 (a) 8 và 20
 (b) 8 và 19
☒ (c) 7 và 19
 (d) 7 và 20
 (e) Tất cả các câu trên đều sai
- [L.O.2] Cho một mảng gồm 7 biến $[x_0, \dots, x_6]$. Giả sử sau quá trình chuyển mảng trên thành min-heap ta được mảng: $[x_6, x_4, x_2, x_3, x_1, x_5, x_0]$ Mảng nào sau đây có thể là kết quả sắp xếp tăng dần của mảng trên?
 (a) $[x_6, x_1, x_2, x_3, x_4, x_5, x_0]$
 (b) $[x_6, x_2, x_5, x_0, x_3, x_4, x_1]$
 (c) $[x_6, x_4, x_3, x_2, x_5, x_0, x_1]$
 (d) $[x_6, x_4, x_1, x_3, x_5, x_2, x_0]$

Dữ liệu dùng cho 2 câu tiếp theo:

Cho cấu trúc dữ liệu của một node trên cây nhị phân như sau:

```
template<typename E>
class BNode {
    virtual E &element() = 0; // return the node
    → value
    virtual BNode<E> *left() const = 0; //return
    → the node's left child
    virtual BNode<E> *right() const = 0; //return
    → the node's right child
};
```

Hãy chọn mã thích hợp để điền vào các chỗ trống trong hàm checkNode để hàm isComplete có thể xác định được cây nhị phân có nút gốc là root có phải là cây hoàn chỉnh (complete) không?

```
template<typename E>
bool checkNode(BNode<E> *node, bool &flag, Queue
→ <BNode<E>> &q) {
    if (node) {
        if (/*Code1*/) return false;
        /*Code2*/;
    } else flag = true;
    return true;
}
```

```
template<typename E>
bool isComplete(BNode<E> *root) {
    if (root == nullptr) return true;
    Queue < BNode<E> * > q;
    q.push(root);
    bool flag = false;
    while (!q.empty()) {
```

- [L.O.2.2] Hoàn thiện đoạn code sau cho hàm insert một giá trị mới vào cây AVL, vị trí trống là lời gọi hàm nào? Các đặc tả khác như trong slide bài giảng và bài thí nghiệm.

```
Node *insert(Node *&subroot, const T &value)
→ {
    Node *pNew = new Node(value);
    if (!subroot) return pNew;
    if (value < subroot->data) {
        subroot->pLeft =
        → insert(subroot->pLeft, value);
    } else if (value >= subroot->data)
        subroot->pRight =
        → insert(subroot->pRight, value);
    else return subroot;
    int balance = getBalance(subroot);
    if (balance > 1 && value <
    → subroot->pLeft->data) return
    → /*Code*/;
}
```

- rotateLeft(subroot)
- rotateLeft(subroot->pLeft)
☒ (c) rotateRight(subroot)
- rotateRight(root)

```

BNode<E> *temp = q.front();
q.pop();
if (!checkNode(temp->left(), flag, q))
    ↪ return false;
if (!checkNode(temp->right(), flag, q))
    ↪ return false;
}

```

4. [L.O.3.3] Điền vào Code1

- (a) flag
- (b) !flag
- (c) q.empty()
- (d) !q.empty()

5. [L.O.3.3] Điền vào Code2

- (a) node = q.front()
- (b) flag = false
- (c) q.push(node)
- (d) q.pop()

6. [L.O.2] Lần lượt chèn vào các giá trị 8 2 12 3 4 7 vào một AVL đang trống. Kết quả duyệt hậu thứ tự (Postorder) của cây sẽ là

- (a) 2 3 7 12 8 4
- (b) 2 3 4 7 8 12
- (c) 3 2 4 8 7 12
- (d) 4 3 2 8 7 12

7. [L.O.2.1] Sai biệt chiều cao tối đa giữa các lá trong cây AVL có thể là bao nhiêu?

- (a) 0 hoặc 1
- (b) Nhiều nhất là 1
- (c) n , trong đó n là số node
- (d) $\log_2 n$ trong đó n là số node
- (e) Không xác định được

8. [L.O.2.2] Cho cây tìm kiếm nhị phân tạo ra bởi việc chèn lần lượt 3, 7, 12, 5, 10, 2, 8. Chèn node 9 vào cây, nó sẽ ở vị trí nào?

- (a) Là node con trái của node 5
- (b) Là node cha của node 8
- (c) Là node con phải của node 8
- (d) Là node con trái của node 10

9. [L.O.2.1] Số nút tối đa của một cây nhị phân có chiều cao h ?

- (a) h
- (b) $2^{h-1} + 1$
- (c) $2^h - 1$
- (d) $2^{h+1} - 1$

10. [L.O.3.1] Làm thế nào để kiểm tra xem một cây nhị phân có phải là cây tìm kiếm nhị phân hay không?

- (a) Thực hiện việc duyệt pre-order và kiểm tra xem kết quả có được sắp xếp hay không
- (b) Thực hiện việc duyệt in-order và kiểm tra xem kết quả có được sắp xếp hay không
- (c) Thực hiện việc duyệt post-order và kiểm tra xem kết quả có được sắp xếp hay không
- (d) Thực hiện việc duyệt breadth-first và kiểm tra xem kết quả có được sắp xếp hay không

11. [L.O.3.1] Dùng giải thuật tìm kiếm nội suy (interpolation search) để tìm vị trí của 48 trong dãy sau:

[1, 5, 8, 11, 19, 22, 31, 35, 40, 45, 48, 49, 50]

Cần phải thực hiện bao nhiêu lần so sánh?

- (a) 2
- (b) 3
- (c) 4
- (d) 5
- (e) Tất cả các đáp án trên đều sai

12. [L.O.2.2] Cho max-heap: 90, 73, 41, 25, 36, 17, 1, 2, 3, 19, 26, 7. Trạng thái heap sau khi xóa 36 khỏi heap:

- (a) 90, 73, 41, 25, 26, 17, 1, 2, 3, 7, 19
- (b) 90, 73, 41, 25, 19, 17, 1, 2, 3, 7, 26
- (c) 90, 73, 41, 25, 26, 17, 1, 2, 3, 19, 7
- (d) 90, 73, 41, 25, 19, 17, 1, 2, 3, 26, 7
- (e) Không thể xóa node trung gian trong heap

13. [L.O.3] Cho hàm sau:

```
int fun1(int arr[], int size, int key, int k)
{
    int s = 0, e = size - 1, mid = s + (e - s) / 2;
    int ans = -1;
    while (s <= e) {
        if (arr[mid] == key) {
            if (arr[mid - k] == arr[mid]) e = mid - 1;
            else if (arr[mid - (k - 1)] == arr[mid]) {
                ans = mid;
                break;
            } else s = mid + 1;
        } else if (key < arr[mid]) e = mid - 1;
        else if (key > arr[mid]) s = mid + 1;
        mid = s + (e - s) / 2;
    }
    return ans;
}
```

Hãy cho biết kết quả xuất ra màn hình sau khi đoạn mã dưới đây được thực thi. int arr[] {1,5,5,11,19,22,22,22,22,45,48,48,48}; cout << fun1(arr, 13, 22, 4);

- (a) 5
- (b) 6
- (c) 7
- (d) 8**
- (e) Tất cả các đáp án trên đều sai

14. [L.O.3.2] Với mỗi dãy khoá dưới đây, lần lượt thêm vào một bảng băm (rỗng) có kích thước là 19, với hàm băm là $h(k) = k \% 19$, dãy khoá nào xảy ra ít đụng độ nhất?

- (a) 0, 19, 57
- (b) 19, 20, 21, 39, 40
- (c) 19, 20, 21, 22, 23, 40, 41, 57
- (d) 19, 20, 21, 22, 23, 46, 47, 57**

15. [L.O.2] Cho một mảng: 1, 3, 5, 4, 6, 11, 10, 8, 7, 13, 15. Đây là mảng đầu ra đại diện cho kết quả sau hai bước heapify mảng trên thành max heap với giải thuật $O(N)$?

- (a) [15,13,11,8,6,5,10,4,7,3,1]
- (b) [1,15,11,8,13,5,10,4,7,3,6]
- (c) [1,3,5,8,15,11,10,4,7,13,6]
- (d) Các đáp án khác đều sai**

16. [L.O.2] Mảng nào sau đây biểu diễn max heap?

- (a) [17,15,13,9,6,10,5,4,8,3,1]
- (b) [17,15,13,9,6,5,10,4,8,3,1]
- (c) [17,15,13,9,6,5,4,10,3,8,1]
- (d) [17,13,15,6,9,4,10,5,3,1,8]

17. [L.O.3] Cho foo3 là giải thuật tìm kiếm nội suy (Interpolation Search)

```
int foo3(int *arr, int n, int x) {
    int i1 = 0, i2 = (n - 1), pos;
    while (i1 <= i2 && x >= arr[i1] && x <= arr[i2]) {
        if (i1 == i2) {
            if (arr[i1] == x) return i1;
            return -1;
        }
        //Line 1
        if (arr[pos] == x) return pos;
        if (arr[pos] < x) i1 = pos + 1;
        else i2 = pos - 1;
    }
    return -1;
}
```

Câu lệnh phù hợp để điền vào vị trí Line 1:

- 1) $pos = i1 + \text{float}(i2-i1) * (x-arr[i1]) / (arr[i2]-arr[i1]);$
- 2) $pos = i1 + \text{float}(i2-i1) * (x-arr[i1]) / (arr[i1]-arr[i2]);$
- 3) $pos = i1 + \text{float}(i2-i1) * (arr[i1]-x) / (arr[i1]-arr[i2]);$
- 4) $pos = i1 + \text{float}(i2-i1) * ((x-arr[i1]) / (arr[i2]-arr[i1]));$**

- (a) Có đúng 1 câu lệnh có thể chọn**
- (b) Có đúng 2 câu lệnh có thể chọn
- (c) Có đúng 3 câu lệnh có thể chọn
- (d) Không có câu lệnh nào phù hợp

18. [L.O.3.1] Cho đồ thị có hướng $G(V_x, E_d)$, $V_x = \{A, B, C, D, E, F\}$ và E_d là ma trận kề $\{\{0, 1, 1, 0, 0, 0\}, \{0, 0, 0, 0, 1, 0\}, \{0, 0, 0, 1, 1, 0\}, \{0, 0, 0, 0, 1, 1\}, \{1, 1, 0, 0, 0, 0\}, \{0, 0, 0, 0, 0, 0\}\}$. Những đỉnh nào được duyệt qua để tìm kiếm đường đi từ A đến F trong Breath-First Search - BFS?

- (a) A E F
- (b) A B D F
- (c) A B C D F
- (d) A B C D E F
- (e) Tất cả các câu trên đều sai

A B C E D F

19. [L.O.3.3] Trong các hiện thực hàm thăm dò hash $hp(k, p, M)$ với k là khoá và M là kích thước bộ nhớ, hiện thực nào tốt nhất để giải quyết đụng độ theo phương pháp đánh địa chỉ mở

- (a) $h(k, p, M) = (a*k + b*p) \% M$, với a, b là hai hằng nguyên tố
- (b) $h(k, p, M) = h_1(k, M) + p*h_2(k, M)$, với h_1, h_2 là hai hàm hash**
- (c) $h(k, p, M) = (h_1(k, M) + p*h_2(k, M)) \% M$, với h_1, h_2 là hai hàm hash
- (d) $h(k, p, M) = \text{rot}(k, p) \% M$, với rot là hàm xoay vòng p lần
- (e) Tất cả các câu trên đều sai

20. [L.O.3.2] Trong thực tế, khi số lượng record trong cơ sở dữ liệu quá nhiều, và mỗi record lại chứa nhiều khoá khác nhau, thì giải pháp nào là phù hợp để phục vụ mục tiêu tìm kiếm trong điều kiện xử lý nhanh trong real-time?

- (a) Sử dụng nhiều cây cân bằng AVL, hoặc Red-Black
- (b) Sử dụng nhiều cây B-Tree hoặc các biến thể cao hơn của nó
- (c) Sử dụng giải thuật băm với khoá là hàm heuristic tổ hợp các khoá
- (d) Sử dụng k-D tree
- (e) Tất cả các câu trên đều sai

21. [L.O.2] Cho một mảng: [10, 7, 11, 5, 4, 13, 1, 2]. Đây là mảng đầu ra đại diện cho min heap được tạo ra bằng cách thêm vào từng phần tử của mảng trên?

- (a) [1, 2, 4, 5, 7, 13, 11, 10]
- (b) [1, 2, 5, 4, 7, 13, 11, 10]
- (c) [1, 2, 5, 4, 7, 11, 13, 10]
- (d) [1, 2, 4, 5, 7, 10, 11, 13]

22. [L.O.2.1] Bất lợi của việc dùng cây Splay?

- (a) Thao tác splay khó
- (b) Không có bất lợi nào đáng kể
- (c) Cây splay thực hiện các bước splay không cần thiết khi một node được đọc
- (d) Chiều cao cây splay có thể tuyến tính khi truy xuất phần tử theo thứ tự tăng dần

23. [L.O.2] Cho min-heap: [1, 2, 6, 4, 7, 13, 11, 10]. Đây là min-heap đầu ra nếu ta thêm vào mảng trên phần tử có giá trị 3?

- (a) [1, 2, 3, 4, 7, 13, 11, 10, 6]
- (b) [1, 2, 6, 3, 7, 13, 11, 10, 4]
- (c) [1, 2, 6, 4, 7, 13, 11, 10, 3]
- (d) [1, 2, 6, 4, 3, 13, 11, 10, 7]

24. [L.O.2.1] Phát biểu nào sau đây là đúng với một heap có độ cao h:

- (a) Phần tử có key lớn nhất sẽ là root của heap
- (b) Các phần tử leaf của heap chỉ nằm ở độ cao h
- (c) Heap là một cây nhị phân hoàn chỉnh (complete binary tree)
- (d) Các phát biểu khác đều đúng

25. [L.O.3] Cho hàm như sau

```
int foo1(int *arr, int idx1, int idx2, int x)
{
    if (idx2 >= idx1) {
        int idx3 = idx1 + (idx2 - idx1) / 2;
        if (arr[idx3] == x) return idx3;
        if (arr[idx3] < x) return foo1(arr,
            ↪ idx1, idx3 - 1, x);
        ↪ return foo1(arr, idx3 + 1, idx2, x);
    }
    ↪ return -1;
}
```

Hãy cho biết kết quả xuất ra màn hình sau khi đoạn mã sau thực thi xong.

```
int arr[]={2, 3, 4, 10, 40};
cout << foo1(arr, 0, 5, 10);
```

- (a) 0
- (b) 3
- (c) 4
- (d) Tất cả các đáp án trên đều sai

26. [L.O.2] Chọn hiện thực đúng của hàm prtIn xuất cây nhị phân theo duyệt cây trung thứ tự (in-order traversal):

- (a)

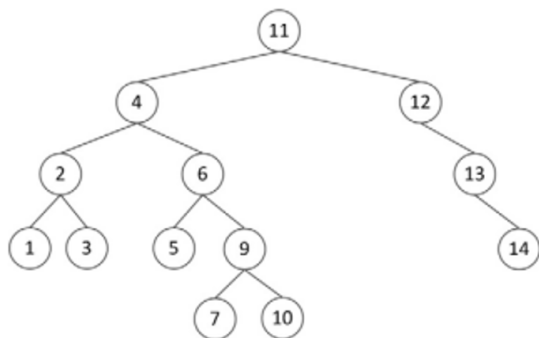
```
if (root != NULL) { cout <<
    root->data; prtIn(root->left);
    prtIn(root->right); }
```
- (b)

```
if (root != NULL) { cout <<
    root->data; prtIn(root->right);
    prtIn(root->left); }
```
- (c)

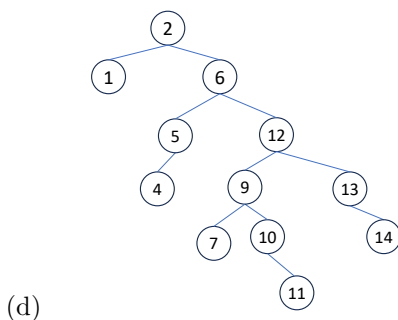
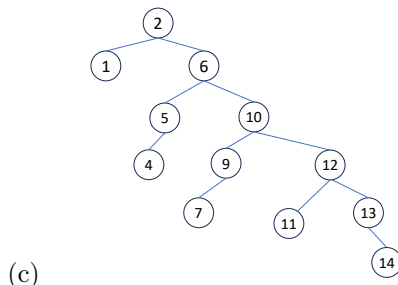
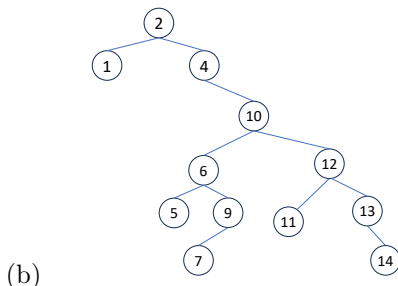
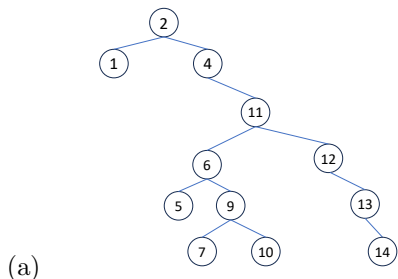
```
if (root != NULL) {
    prtIn(root->left); cout << root->data;
    prtIn(root->right); }
```
- (d)

```
if (root != NULL) {
    prtIn(root->right); prtIn(root->left);
    cout << root->data; }
```

27. [L.O.3.1] Cho cây Splay sau:



Thực hiện xoá khoá 3 với giả thuyết đây là cây splay bottom-up



28. [L.O.2.1] Với cây tìm kiếm nhị phân, thứ tự duyệt trước (pre-order) là:

- (a) Node – Left – Right
- (b) Left – Node – Right
- (c) Right – Node – Left
- (d) Right – Left – Node

29. [L.O.3.1] Cho hàm hash như sau: $h(k, M) = \text{rot}(k, 2) \% M$, trong đó $\text{rot}(k, m)$ là phép quay sang trái m chữ số, M là kích thước bộ nhớ. Hãy cho biết kết quả của đoạn lệnh sau: `vector<int> v{1026, 1023, 1022, 1025, 1001};`

`for (auto val:v) cout << h1(val, 107) << ' ';`

- (a) 26 23 22 25 11
- (b) 42 63 70 49 3
- (c) 61 31 21 51 10
- (d) 610 310 210 510 011
- (e) Tất cả các câu trên đều sai

30. [L.O.1] Độ phức tạp thời gian trong trường hợp xấu nhất khi chèn n^2 phần tử vào cây AVL ban đầu có n phần tử?

- (a) $O(n \log_2 n)$
- (b) $O(n^2 \log_2 n)$
- (c) $O(n^3)$
- (d) $O(n^3 \log_2 n)$

31. [L.O.1] Chọn phát biểu đúng

- (a) Cây tìm kiếm nhị phân luôn cho phép tìm kiếm với độ phức tạp $O(\log_2 N)$
- (b) Cây Heap cho phép tìm kiếm với độ phức tạp $O(\log_2 N)$
- (c) Giải thuật Hashing cho phép tìm kiếm với độ phức tạp $O(1)$
- (d) Ba phát biểu trên đều đúng
- (e) Các phát biểu trên đều sai

32. [L.O.3.3] Cho hàm chèn node vào cây tìm kiếm nhị phân, trả về gốc của cây.

```
TreeNode *bstInsert(TreeNode *root, int val)
↪ {
    if (root == nullptr) { return new
        ↪ TreeNode(val); }
    TreeNode *curr = root;
    TreeNode *prev = nullptr;
    while (curr != nullptr) {
        prev = curr;
        // Code
    }
    if (val < prev->val) { prev->left = new
        ↪ TreeNode(val); } else { prev->right =
        ↪ new TreeNode(val); }
    return root;
}
```

Không chấp nhận khoá trùng trên cây, điền vào Code để hoàn thành hàm trên

- (a) if (val < curr->val) curr = curr->left; else curr = curr->right;
- (b) if (val < curr->val) curr = curr->left; else if (val > curr->val) curr = curr->right; else break;
- (c) if (val < curr->val) curr = curr->left; else if (val > curr->val) curr = curr->right;
- (d) if (val < curr->val) curr = curr->left; else if (val > curr->val) curr = curr->right; else return root;
- (e) Tất cả các đáp án trên đều sai

33. [L.O.2.2] Để hiện thực một giải thuật Depth-First Search trên một đồ thị, cấu trúc dữ liệu gì được sử dụng?

- (a) Hàng (Queue)
- ☒ (b) Chồng (Stack)
- (c) Heap
- (d) Danh sách liên kết (Linked List)

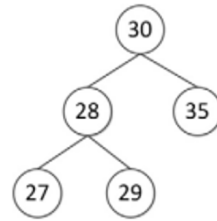
34. [L.O.2.1] Trong cây nhị phân, độ sâu (depth) của một node là:

- (a) Số lượng nút con trực tiếp của nút đó
- (b) Số lượng nút cha trực tiếp của nút đó
- (c) Số lượng nút con trực tiếp và gián tiếp của nút đó
- (d) Số lượng nút cha trực tiếp và gián tiếp của nút đó

35. [L.O.2.1] Cấu trúc dữ liệu nào sau đây được dùng khi duyệt một đồ thị theo chiều rộng?

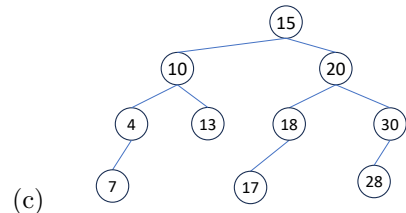
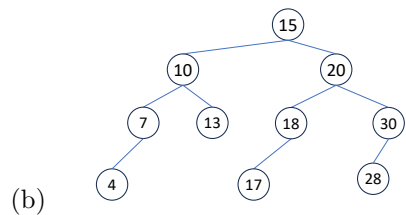
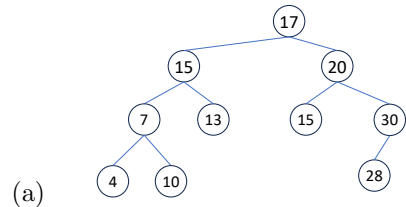
- (a) List
- ☒ (b) Queue
- (c) Stack
- (d) Phương án khác

36. [L.O.2] Cho cây bên dưới:



Sau khi xoá 35 và rồi chèn

17 vào:



- (d) Tất cả các đáp án trên đều sai

37. [L.O.2] Lần lượt chèn vào các giá trị 2 8 12 3 4 vào một AVL đang trống. Kết quả duyệt tiền thứ tự (Preorder) của cây sẽ là

- (a) 8 2 3 4 12
- (b) 2 4 3 12 8
- (c) 8 3 12 2 4
- (d) 8 3 2 4 12

38. [L.O.3.1] Làm thế nào để tìm tổ tiên chung thấp nhất (lowest common ancestor - LCA) của hai nút trong một cây tìm kiếm nhị phân?

- (a) Bắt đầu từ gốc và duyệt cây cho đến khi cả hai nút đều ở cùng một phía của nút hiện tại
- (b) Bắt đầu từ gốc và duyệt cây cho đến khi cả hai nút bằng với nút hiện tại
- (c) Bắt đầu từ gốc và duyệt cây cho đến khi cả hai nút nhỏ hơn hoặc lớn hơn nút hiện tại
- (d) Bắt đầu từ gốc và duyệt cây cho đến khi cả hai nút ở hai phía khác nhau của nút hiện tại hoặc một trong số chúng là nút hiện tại

39. [L.O.3] Cho hàm foo2 hiện thực một giải thuật tìm kiếm:

```
int foo2(int arr[], int x, int n) {
    int step = sqrt(n);
    int prev = 0;
    // Code here
    while (arr[prev] < x) {
        prev++;
        if (prev == min(step, n)) return -1;
    }
    if (arr[prev] == x) return prev;
    return -1;
}
```

Phải điền vào vị trí (// Code here) đoạn mã nào:

- (a) while (arr[step-1] < x){prev = step; step += sqrt(n);}
- (b) while (arr[min(step, n)-1] < x){prev = step; step += sqrt(n);}
- (c) while (arr[min(step, n)-1] < x){step += sqrt(n);}
- (d) Tất cả các đáp án trên đều sai

40. [L.O.2] Chèn lần lượt các key sau vào một cây B-Tree (m=3) rỗng: 50, 40, 70, 55, 45, 43, 56, 58. Trong cây trên có bao nhiêu node có nhiều hơn 1 entry?

- (a) 0
- (b) 1
- (c) 2
- (d) 3

41. [L.O.2.2] Cho cây tìm kiếm nhị phân, tiến hành chèn các giá trị lần lượt là 5, 10, 15, 20, 30, 35, 40. Chèn 27 vào cây, ở độ sâu 3 (root ở độ sâu 0) ta thấy node:

- (a) 20
- (b) 27
- (c) 30
- (d) 35

42. [L.O.3.2] Cho một tập điểm trong không gian 3D (x,y,z) với số lượng lớn. Giải pháp nào sau đây là tối ưu nhất cho bài toán trên

- (a) Cây tìm kiếm nhị phân cân bằng với khoá là $x*y*z$
- (b) Cây B-Tree với khoá là $x*y*z$
- (c) Bảng Hash với khoá chia theo bin
- (d) Cây k-D với $k = 3$

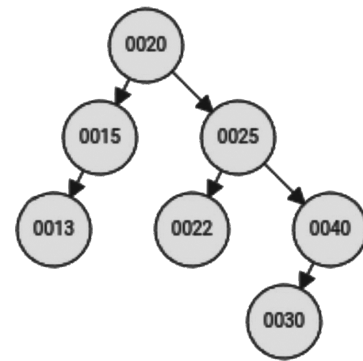
43. [L.O.2] Để hiện thực giải thuật sắp xếp đồ thị (topology sort), một giá trị đếm sẽ được gán với mỗi đỉnh của đồ thị. Hãy chọn giá trị đếm phù hợp cho đỉnh b sau khi đã sắp xếp được 2 đỉnh trong đồ thị sau

- (a) 0
- (b) 1
- (c) 2
- (d) 3

44. [L.O.3.1] Cho đồ thị có hướng $G(V_x, E_d)$, $V_x = \{A, B, C, D, E, F\}$ và E_d là ma trận kề $\{\{0, 1, 1, 0, 1, 0\}, \{1, 0, 0, 1, 1, 0\}, \{1, 0, 0, 1, 1, 0\}, \{0, 1, 1, 0, 1, 0\}, \{1, 1, 1, 1, 0, 0\}, \{0, 0, 0, 0, 0, 0\}\}$. Kết quả xuất đồ thị trong tìm kiếm theo chiều sâu (Depth-First Search – DFS) là gì?

- (a) A B C D E F
- (b) A B D C E F
- (c) A C D B E F
- (d) A E D C E F
- (e) Tất cả các câu trên đều sai

45. [L.O.2] Cho AVL như hình vẽ, hãy cho biết in ra theo tiền thứ tự (Preorder) của cây AVL sau khi xóa 13 khỏi cây



- (a) 20 15 25 22 40 30
- (b) 20 25 15 40 22 30
- (c) 25 20 15 22 40 30
- (d) 25 20 22 15 40 30

46. [L.O.1.1] Độ phức tạp của các giải thuật xây dựng heap như heapify và bottom-up-build tương ứng là:

- (a) $O(\log_2 n)$ và $O(n)$
- (b) $O(n)$ và $O(\log_2 n)$
- (c) $O(n)$ và $O(n)$
- (d) $O(\log_2 n)$ và $O(\log_2 n)$
- (e) Tất cả các câu trên đều sai

47. [L.O.3.3] Cho hàm sau:

```
int countL0(Node *root) {
    if (!root == NULL) return 0;
    // Code
    return countL0(root->left) +
        ↪ countL0(root->right) + 1;
}
```

Điền vào Code để hoàn thành hàm trả về số node lá trong cây nhị phân.

- (a) if (!root->left && !root->right) return 1;
- (b) if (!root->left | !root->right) return 1;|
- (c) if (!root->left && !root->right) return 0;
- (d) if (!root->left | !root->right) return 0;|
- (e) Tất cả các đáp án trên đều sai

Dữ liệu sau cho 2 câu tiếp theo:

```
class Heap {
    int *harr;// pointer to array of elements in
    ↪ heap
    int heap_size;// Current number of elements.
public:
    bool isMinHeap() { // This function will return
    ↪ true if given level order traversal is Min
    ↪ Heap
    for (int i = (heap_size / 2 - 1); i >= 0;
    ↪ i--) {
        if (/*Code1*/)
            return false;
        if (/*Code2*/) {
            if (harr[i] > harr[2 * i + 2])
                return false;
        }
    }
    return true;
}
}
```

48. [L.O.3.3] Điền vào Code1:

- (a) harr[i] < harr[2 * i + 1]
- (b) harr[i] > harr[2 * i + 2]
- (c) harr[i] < harr[2 * i + 2]
- (d) harr[i] > harr[2 * i + 1]
- (e) Tất cả các câu trên đều sai

49. [L.O.3.3] Điền vào Code2:

- (a) 2*i + 1 < heap_size
- (b) 2*i + 2 < heap_size
- (c) 2*i + 2 <= heap_size
- (d) 2*i + 1 == heap_size
- (e) Tất cả các câu trên đều sai

50. [L.O.3.1] Giả sử các số 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 theo thứ tự được chèn vào cây tìm kiếm nhị phân (BST) rỗng ban đầu. Cây nhị phân tìm kiếm sử dụng thứ tự thông thường trên các số tự nhiên. Kết quả duyệt cây trung thứ tự là gì?

- (a) 0 1 2 3 4 5 6 7 8 9
- (b) 0 2 4 3 1 6 5 9 8 7
- (c) 7 5 1 0 3 2 4 6 8 9
- (d) 9 8 6 4 2 3 0 1 5 7
- (e) Tất cả các câu trên đều sai

51. [L.O.2.2] Cho AVLTree được mô tả như trong các bài thí nghiệm, hãy điền vào chỗ trống để hoàn thiện đoạn code chèn nhiều phần tử (theo thứ tự) từ một mảng vào cây AVL: AVLTree<int> avl;

```
int arr[] = {20, 18, 40, 4, 5, 50, 17, 28};
for (int i = 0; i < 8; i++){
    -----;}
```

- (a) avl.insert(arr[i])
- (b) avl->insert(arr[i])
- (c) *avl.insert(arr[i])
- (d) (*avl)->insert(arr[i])

Dữ liệu cho 5 câu hỏi tiếp theo:

Cho lớp Graph được hiện thực dùng ma trận liên kề như sau:

```
class Graph {
private:
    int nover; //số đỉnh trong đồ thị
    int **wm; // ma trận trọng số
    int minDistance(int dist[], bool sptSet[]) {
        int min = INT_MAX, min_index;
        for (int v = 0; v < nover; v++)
            if (sptSet[v] == false && dist[v] <=
                ↪ min)
                min = dist[v], min_index = v;
        return min_index;
    }
}
```

Hãy chọn lựa đoạn mã thích hợp trong các chỗ trống trong đoạn mã sau để hoàn thành phương thức hiện thực giải thuật Dijkstra:

```
void dijkstra(int src) {
    int dist[nover];
    bool sptSet[nover];
    for (int i = 0; i < nover; i++)
        dist[i] = INT_MAX, sptSet[i] = false;
    dist[src] = 0;
    for (int count = 0; count < /*Code1*/;
        ↪ count++) {
        int u = minDistance(dist, sptSet);
        /*Code2*/ = true;
        for (int v = 0; v < nover; v++)
            if (!sptSet[v] && dist[u] !=
                ↪ INT_MAX && /*Code3*/)
                dist[v] = /*Code4*/;
    }
    //printSolution(dist);
}
```

52. [L.O.3.3] Điền vào Code1:

- (a) `nover`
- (b) `nover - 1`
- (c) `src`
- (d) `src - 1`

53. [L.O.3.3] Điền vào Code2

- (a) `sptSet[u]`
- (b) `sptSet[src]`
- (c) `sptSet[count]`
- (d) `sptSet[nover]`

54. [L.O.3.3] Điền vào Code3

- (a) `wm[u][v] < dist[v]`
- (b) `dist[v] + wm[u][v] < dist[u]`
- (c) `wm[u][v] < dist[u]`
- (d) `dist[u] + wm[u][v] < dist[v]`

55. [L.O.3.3] Điền vào Code4

- (a) `dist[u]`
- (b) `wm[u][v]`
- (c) `dist[u] + wm[u][v]`
- (d) `dist[v] + wm[u][v]`

56. [L.O.3.3] Với chức năng trả về vị trí của đỉnh có trọng số nhỏ nhất và sau đó loại bỏ đỉnh này khỏi danh sách ở mỗi lần lặp, nên áp dụng cấu trúc dữ liệu nào cho hàm `minDistance` thì thích hợp nhất để giảm độ phức tạp thời gian thực thi?

- (a) Đống (Heap)
- (b) Chồng (Stack)
- (c) Danh sách liên kết đơn (Singly Linked List)
- (d) AVL

57. [L.O.2.1] Số node tối thiểu của một cây nhị phân có chiều cao h ?

- (a) h
- (b) $h + 1$
- (c) 2^{h-1}
- (d) $2^{h-1} + 1$

58. [L.O.3.3] Khoá cần hash là một chuỗi ký tự. Hàm hash được định nghĩa như sau: tổng của ký tự hiện tại nhân với phần dư (khi chia 7) của ký tự ngay phía sau nó, tất cả lấy phần dư chia cho M , với M là kích thước bộ nhớ. Cho hàm `int h2(string str, int M)`, hiện thực bên trong là:

- (a) `int acc = 0; for (char c:str) acc += c; return acc % M;`
- (b) `for (char c:str) c += c; return c % M;`
- (c) Hai đáp án trên đều sai
- (d) Hai đáp án trên đều đúng

59. [L.O.3.1] Dùng giải thuật tìm kiếm nhị phân (binary search) để tìm vị trí của 48 trong dãy sau:

[1,5,8,11,19,22,31,35,40,45,48,49,50]

Cần phải thực hiện bao nhiêu lần so sánh?

- (a) 2
- (b) 3
- (c) 4
- (d) 5
- (e) Tất cả các đáp án trên đều sai

60. [L.O.1.2] Chiều cao tối đa của AVL với p node?

- (a) p
- (b) $p/2$
- (c) $\log_2(p)$
- (d) $\log_2(p)/2$
- (e) Tất cả các đáp án trên đều sai